Shoepz E-Commerce API Development

> **Project Overview:** This document outlines the step-by-step development and explanation of the backend architecture for the "Shoepz" e-commerce platform based on Node.js, Express, and MongoDB, according to the provided requirements.

1 & 3. Setup Express Server & Middleware

The first step involves initializing a Node.js project and setting up the Express application with essential middleware like `cors` for cross-origin requests and JSON body parsing.

```
// server.js (Entry Point)
const express = require('express');
const cors = require('cors');
const dotenv = require('dotenv');
const connectDB = require('./config/db');

dotenv.config();

// Connect to Database
connectDB();

const app = express();

// Middleware
app.use(cors());
app.use(express.json()); // Built-in body-parser alternative

// Routes (To be defined)
app.use('/api/users', require('./routes/userRoutes'));
app.use('/api/products', require('./routes/productRoutes'));
app.use('/api/orders', require('./routes/orderRoutes'));

// Global Error Handler
app.use(require('./middlewares/errorHandler'));

const PORT = process.env.PORT || 5000;
app.listen(PORT, () => console.log(`Server running on port ${PORT}`));
```

2. Database Configuration

We use Mongoose to establish a connection to a MongoDB database (either locally with Compass or cloud-based with Atlas).

```
// config/db.js
const mongoose = require('mongoose');

const connectDB = async () => {
    try {
        const conn = await mongoose.connect(process.env.MONGO_URI, {
            useNewUrlParser: true,
            useUnifiedTopology: true
        });
        console.log(`MongoDB Connected: ${conn.connection.host}`);
    } catch (error) {
        console.error(`Error: ${error.message}`);
        process.exit(1);
    }
};
module.exports = connectDB;
```

5. Implement Data Models

We define Mongoose schemas for Users, Products, and Orders to enforce structure on the NoSQL collections.

User Model

```
// models/User.js
const mongoose = require('mongoose');

const userSchema = mongoose.Schema({
    name: { type: String, required: true },
    email: { type: String, required: true, unique: true },
    password: { type: String, required: true },
    isAdmin: { type: Boolean, required: true, default: false }
}, { timestamps: true });

module.exports = mongoose.model('User', userSchema);
```

Product Model

```
// models/Product.js
const mongoose = require('mongoose');

const productSchema = mongoose.Schema({
    user: { type: mongoose.Schema.Types.ObjectId, required: true, ref: 'User' }, // Admin
    name: { type: String, required: true },
    image: { type: String, required: true },
    brand: { type: String, required: true },
    category: { type: String, required: true },
    description: { type: String, required: true },
    price: { type: Number, required: true, default: 0 },
    countInStock: { type: Number, required: true, default: 0 }
}, { timestamps: true });

module.exports = mongoose.model('Product', productSchema);
```

4 & 6. Define API Routes & User Authentication

Authentication utilizes JSON Web Tokens (JWT) for stateless sessions and bcrypt for password hashing. We use middleware to protect private routes.

```
// routes/userRoutes.js
const express = require('express');
const router = express.Router();
const { registerUser, loginUser, logoutUser } = require('../controllers/userController');

router.post('/register', registerUser);
router.post('/login', loginUser);
router.post('/logout', logoutUser); // Usually handled client-side by dropping token, but
```

```
module.exports = router;
```

Authentication and Authorization Middleware:

```
// middlewares/authMiddleware.js
const jwt = require('jsonwebtoken');
const User = require('../models/User');

const protect = async (req, res, next) => {
    let token;
    if (req.headers.authorization && req.headers.authorization.startsWith('Bearer')) {
        try {
            token = req.headers.authorization.split(' ')[1];
            const decoded = jwt.verify(token, process.env.JWT_SECRET);
            req.user = await User.findById(decoded.id).select('-password');
            next();
        } catch (error) {
            res.status(401);
            throw new Error('Not authorized, token failed');
        }
    }
    if (!token) {
        res.status(401);
        throw new Error('Not authorized, no token');
    }
};

const admin = (req, res, next) => {
    if (req.user && req.user.isAdmin) {
        next();
    } else {
        res.status(401);
        throw new Error('Not authorized as an admin');
    }
};

module.exports = { protect, admin };
```

7 & 8. Handle Products, Orders & Admin Functionality

Creating controllers and routes to handle public product fetching, authenticated order creation, and admin-only management (adding products/managing orders).

```
// routes/productRoutes.js
const express = require('express');
const router = express.Router();
const { getProducts, createProduct } = require('../controllers/productController');
const { protect, admin } = require('../middlewares/authMiddleware');

// getProducts is public, createProduct requires user to be logged in AND be an admin
router.route('/').get(getProducts).post(protect, admin, createProduct);

module.exports = router;
```

9. Error Handling

A global error handling middleware guarantees clients receive proper HTTP status codes and standard JSON error payloads instead of HTML stack traces.

```
// middlewares/errorHandler.js
const errorHandler = (err, req, res, next) => {
    const statusCode = res.statusCode === 200 ? 500 : res.statusCode;
    res.status(statusCode);
    res.json({
        message: err.message,
        stack: process.env.NODE_ENV === 'production' ? null : err.stack,
    });
};

module.exports = errorHandler;
```

Conclusion

This backend architecture follows standard RESTful principles using the Node.js/Express/MongoDB stack. It provides a secure, scalable foundation for the Shoepz application, effectively segregating concerns into models, controllers, routes, and middleware.